# IoT Cat Feeder

An automatic, timed cat feeder with a web dashboard on the ESP8266

## Project Report

Chindanai Jaiman 662115012
Bachelor of Science Software Engineering Program
College of Arts, Media, and Technology, Chiang Mai University

November 2026

Project Advisor
**Asst.Prof.Phudinan Singkhamfu, Ph.D.**

# Table of Contents

**Abstract** ..... **4**
**Part I Project Proposal** ..... **5**
**Chapter 1 Introduction** ..... **5**
1.1 Motivation ..... 5
1.2 Purpose ..... 5
1.3 Target Users and Intended Experience ..... 5
1.4 Project Scope ..... 6
1.5 Objectives ..... 6
**Chapter 2 Review and Analysis** ..... **7**
2.1 Tool and Technology Review ..... 7
2.2 Risk Analysis ..... 7
2.3 Estimated Cost ..... 8
**Part II Software Requirements Specification** ..... **9**
**Chapter 3 Introduction** ..... **9**
3.1 Purpose ..... 9
3.2 Scope ..... 9
3.3 Product Perspective ..... 9
3.4 Product Functions ..... 9
3.5 User Characteristics ..... 10
3.6 Limitations ..... 10
3.7 Assumptions and Dependencies ..... 10
3.8 Definitions ..... 11
3.9 References ..... 11
**Chapter 4 Specification Requirements** ..... **12**
4.1 Feature Description ..... 12
4.2 Use Case Diagram ..... 13
4.3 Use Case Description ..... 13
4.4 Activity Diagram ..... 15
4.5 Functional Requirements ..... 16
4.6 External Interface Requirements ..... 17
4.7 Data Requirements ..... 18
4.8 Quality Requirements ..... 18
4.9 Design Constraints ..... 19
4.10 Verification ..... 19
**Part III Software Design Description** ..... **20**
**Chapter 5 Introduction** ..... **20**
5.1 Purpose ..... 20
5.2 Scope ..... 20
5.3 Design Overview ..... 20
5.4 Definitions ..... 20
**Chapter 6 System Architecture** ..... **21**
6.1 Architectural Overview ..... 21
6.2 Context View ..... 21

6.3 Component View....22
6.4 Deployment and Power Design....22
6.5 Design Rationale....23
**Chapter 7 Detailed Design....24**
7.1 Data Design....24
7.2 Component Description....24
7.3 State Design....25
7.4 Interaction Flows....26
7.5 Interface Design....26
7.6 Error Handling and Recovery....27
7.7 Security Considerations....27
**Chapter 8 Traceability and Verification....28**
8.1 Requirements Traceability Matrix....28
8.2 Test Plan....28
**Part IV Output and Discussion....29**
**Chapter 9 The Delivered System....29**
9.1 Hardware on the bench....29
9.2 Firmware behaviour as delivered....29
9.3 Configuration in the delivered firmware....30
**Chapter 10 Test Results....31**
10.1 Procedure....31
10.2 Measured results....31
10.3 Tests not run....31
10.4 Photos and Evidence....31
**Chapter 11 Discussion....36**
11.1 Clock and schedule....36
11.2 Flap safety....36
11.3 Presence detection....36
11.4 Hopper level....36
11.5 The portion control decision....36
11.6 Defects found and fixed....37
11.7 Limitations and threats to validity....37
**Part V Conclusion and Future Work....38**
**Chapter 12 Conclusion....38**
12.1 Summary....38
12.2 Against the objectives....38
12.3 Lessons....38
12.4 Limitations accepted in this version....39
**Chapter 13 Future Work....40**
13.1 Portion control by weight....40
13.2 Persistence and notification....40
13.3 Cat identity and imaging....40
13.4 Reliability and enclosure....40

## Abstract

This report documents an automatic dry-food feeder for one cat, built on a NodeMCU v3 with one servo and two ultrasonic sensors, and serving its own dashboard over the local Wi-Fi network. The problem is that an owner who is out of the house cannot deliver the midday meal, and a bowl left full all day removes portion control. The delivered device dispenses at 08:00, 13:00, and 18:00 from network time, feeds on demand from a phone, closes the flap from the main loop so a dropped client cannot strand it, and reports the flap state, the daily count, and the last meal with its trigger source. Portion control by weight and durable history were removed from version 1.0.0 and recorded as future work. The report states the measured results and states plainly which tests were not run and why.

# Part I Project Proposal

*Problem, objectives, scope, and the review that supports them.*

# Chapter 1 Introduction

## 1.1 Motivation

A cat that is fed by hand eats at the convenience of the owner, not at a fixed time. When the owner leaves for work in the morning and returns in the evening, the usual workaround is to pour a large portion into the bowl and let the cat graze all day. Two problems follow. The food sits in the open bowl for ten hours or more, and the cat eats the entire portion before noon and then begs for the rest of the day.

The second problem is portion control. Dry food is calorie dense, and a cat that free feeds commonly gains weight. Veterinary guidance for an adult cat of average weight is a measured daily ration split across several meals, not an open bowl. An owner who is not at home at lunchtime cannot deliver the middle meal.

Commercial automatic feeders solve the timing problem, but most of them depend on a cloud service and a mobile app to program the schedule. If the vendor changes the app, or the household Wi-Fi is replaced, the device becomes a plastic box. For a student project the interesting engineering question is whether the same behaviour can be produced on a microcontroller that the owner can read, repair, and reflash.

## 1.2 Purpose

This project builds a cat feeder that dispenses dry food at three fixed times per day and also on demand from a phone. The device serves its own web dashboard over the local Wi-Fi network, so no account, no cloud service, and no vendor app are required. The owner opens a browser, sees the current state of the flap and the number of meals served today, and can trigger a feeding with one tap.

The project is also a study of the boundary between software and hardware. The interesting part is not the web page. It is the electrical and timing behaviour that the web page depends on: a servo that stalls and browns out the board, a clock that is wrong until the network answers, and a flap that must never stay open.

## 1.3 Target Users and Intended Experience

Two users with different needs must both be served without one damaging the other.

### 1.3.1 Persona: The owner

**Table 1.** Owner persona

| Attribute | Detail |
|---|---|
| Name | Nok |
| Age | 29 |
| Occupation | Office worker, Chiang Mai |
| Background | Lives in a rented room with one adult cat. Leaves at 07:30 and returns at 18:30. Uses a phone for everything. Has no tools beyond a screwdriver. |
| Goals | Feed the cat on a predictable schedule. Check from work that the morning meal was served. Add an extra meal when the cat begs. |
| Pain points | Cannot come home at lunch. Forgets whether the morning meal was served. Does not want another app account, and does not want to learn a configuration language. |

| Attribute | Detail |
|---|---|
| Experience requirement | Open one page, read five values, tap one button. No login, no setup wizard, no pairing step. |

### 1.3.2 Persona: The cat

**Table 2.** Cat persona and constraints

| Attribute | Detail |
|---|---|
| Subject | One adult domestic cat, 4 kg |
| Behaviour | Eats at the sound of the flap. Approaches the device when it is near the bowl. |
| Constraint | The cat must not be able to open the flap with a paw, and must not be trapped by it. |
| Experience requirement | A meal arrives without the cat having to touch anything. The flap cycle is short enough not to startle the cat away from the bowl. |

## 1.4 Project Scope

The scope is set so that the work is complete and demonstrable on 4 November 2026. Features that cannot be finished and tested by 2 November are moved to Future Work rather than shipped half-built.

**Table 3.** Scope of the delivered system

| In scope | Out of scope |
|---|---|
| Scheduled feeding at three times per day using NTP time | Cloud storage and a hosted dashboard |
| On-demand feeding from a browser on the same network | A native mobile application |
| Flap status, daily meal count, last meal time and source | User accounts and authentication |
| Automatic flap close after a fixed open interval | Wet food dispensing |
| Cat presence detection with an ultrasonic sensor (target) | Camera or two-way audio |
| Hopper level from a downward ultrasonic sensor | Portion control by weight with a load cell |
| | Support for more than one feeder or more than one cat |

Portion control left the scope on 2 November, when the header pins of the amplifier were found to be unsoldered. It moved to Future Work rather than ship half-built. Presence detection is wired and integrated, and it ships if it passes the test on 3 November. A requirement that is written and tested earns credit, while a half-connected sensor does not.

## 1.5 Objectives

1. Deliver food at 08:00, 13:00, and 18:00 local time, using time obtained from a network time server rather than a countdown timer.
2. Serve a dashboard from the ESP8266 itself, reachable from a phone on the same Wi-Fi network in under two seconds.
3. Close the flap automatically after every feeding, in all code paths, including the path where the browser disconnects immediately after sending the command.
4. Record the time, count, and trigger source of each meal so the owner can confirm a feeding happened.
5. Detect a cat within 60 cm and dispense a meal in under two seconds, at most once per 30 minutes.
6. Document the design so another student can rebuild the device from the repository alone.

# Chapter 2 Review and Analysis

## 2.1 Tool and Technology Review

Each decision below was made against at least one alternative. The rejected option is recorded so the reader can see the trade that was accepted.

**Table 4.** Technology decisions

| Decision | Chosen | Rejected | Rationale |
|---|---|---|---|
| Microcontroller | NodeMCU v3, ESP8266 | Arduino Uno with ESP-01, ESP32 | Already owned. Has Wi-Fi and a 3.3 V regulator on board. The ESP32 is better but adds cost and no requirement needs it. |
| Actuator | Servo SG90 | Stepper motor, solenoid | The servo holds an angle without power cycling and needs one PWM pin. A solenoid slams and needs a driver transistor. |
| Time source | SNTP over Wi-Fi | DS3231 real-time clock | No extra part, no battery, and the time is correct after every boot. An RTC drifts and needs a battery holder. |
| Transport | HTTP on the device | MQTT broker, Blynk | HTTP needs no broker and no account. Blynk hides the implementation behind a vendor widget, which removes the work the assessment is looking for. |
| Dashboard | Embedded HTML and JavaScript in PROGMEM | SPIFFS files, external CDN | One flash image, no filesystem, no internet dependency, and the page cannot lose its assets. |
| Presence sensing | HC-SR04 ultrasonic | FC-51 infrared proximity | Ultrasonic reports distance, which supports both presence and level. The FC-51 is binary and is fooled by a cardboard hopper wall. |
| Portion control | Open-loop flap timing | HX711 with 1 kg load cell | The load cell measures the actual mass, but the amplifier header pins were not soldered before the deadline. Open-loop timing ships, and the closed loop moves to Future Work. |

## 2.2 Risk Analysis

**Table 5.** Risk register

| ID | Risk | Likelihood | Impact | Mitigation |
|---|---|---|---|---|
| R-01 | Servo stall browns out the board and reboots it mid-meal | High | High | Power the servo from the powerbank rail, not from 3V3. Add a 470 uF capacitor across VIN and GND. Test the worst case with servo_test.ino before integrating. |
| R-02 | Flap stays open and the hopper empties | Medium | High | The close is driven from millis() in the main loop, not from a network callback, so a dropped client cannot strand the flap. |
| R-03 | Clock is wrong or jumps backward after a resync | Medium | Medium | Ignore the schedule until the clock passes a fixed epoch. A duplicate meal inside the same minute is prevented by a minute key. |
| R-04 | Ultrasonic readings oscillate on a kibble surface | High | Medium | Sample ten times per measurement and use the median. Reject echoes outside the hopper depth. |
| R-05 | Wi-Fi in the examination room blocks the device | Low | High | The dashboard is reachable in the local network, so the demonstration does not depend on internet access. A phone hotspot is the fallback. |

## 2.3 Estimated Cost

Prices are estimates from general retail listings and are not yet confirmed with the supplier.

**Table 6.** Estimated bill of materials

| Item | Quantity | Unit (baht) | Total (baht) |
|---|---|---|---|
| NodeMCU v3 (already owned) | 1 | 0 | 0 |
| Servo SG90 (already owned) | 1 | 0 | 0 |
| Capacitor 470 uF 16 V | 1 | 10 | 10 |
| HC-SR04P ultrasonic sensor | 2 | 80 | 160 |
| HX711 amplifier with 1 kg load cell | 1 | 160 | 160 |
| Breadboard, jumpers, resistor (already owned) | 1 set | 0 | 0 |
| Cardboard, tape, food-safe bowl | 1 set | 50 | 50 |
| Powerbank 5 V 2 A (already owned) | 1 | 0 | 0 |
| **Total** | | | **380** |

# Part II Software Requirements Specification

*Functions, use cases, interfaces, and verifiable requirements.*

# Chapter 3 Introduction

## 3.1 Purpose

This document specifies the requirements for the IoT Cat Feeder, version 1.0.0. It is written for the project advisor as the acceptance basis, for the developer as the input to the design, and for the examiner as the evidence that each requirement has a stated, checkable condition.

The document follows the structure of ISO/IEC/IEEE 29148 for software requirements. The section order is adapted to the size of the project: the standard separates product functions from detailed requirements, which this document keeps, and it separates verification from the requirement text, which this document keeps as a matrix in Section 4.10.

## 3.2 Scope

The product is a single-board appliance that dispenses dry cat food on a schedule and on demand, and reports its state through a web page served by the same board.

**Table 7.** System identification

| Item | Value |
|---|---|
| Product name | IoT Cat Feeder |
| Version specified | 1.0.0 |
| Platform | NodeMCU v3, ESP8266EX, 80 MHz, 4 MB flash |
| Runtime | Arduino core for ESP8266, single-threaded main loop |
| Network role | Wi-Fi station client with an embedded HTTP server on port 80 |
| Users | One owner, one cat |
| Operating environment | Indoor, 2.4 GHz Wi-Fi, 5 V 2 A supply |

The product does not store data outside the device, does not authenticate users, and does not depend on any external service other than a network time server.

## 3.3 Product Perspective

The device is a standalone system. It replaces the manual act of pouring food, not a software component inside another product. The owner is the only human actor. The clock is an external service that provides the time reference. The cat is an environmental actor whose position becomes an input through the presence requirement.

## 3.4 Product Functions

**Table 8.** Product functions

| ID | Function | Summary | Status |
|---|---|---|---|
| F1 | Scheduled feeding | Dispenses one meal at each configured time, using network time. | Delivered |
| F2 | On-demand feeding | Dispenses one meal when the owner presses the dashboard button. | Delivered |

| ID | Function | Summary | Status |
|---|---|---|---|
| F3 | Status and history | Reports flap state, daily count, last meal, and link quality. | Delivered |
| F4 | Presence-triggered feeding | Dispenses a meal when a cat is detected near the bowl. | Integrated |
| F5 | Hopper level monitoring | Reports the remaining food as a percentage of hopper depth. | Not delivered |

## 3.5 User Characteristics

**Table 9.** User classes

| User class | Technical level | Characteristics and needs |
|---|---|---|
| Owner | Non-technical | Uses a phone browser. Expects a single page with a single action. Cannot be asked to install software, create an account, or read a manual. |
| Maintainer | Technical | Rebuilds and reflashes the device. Needs the wiring, the pin map, the library list, and the credentials file documented. |
| Cat | None | Interacts physically. Must not be able to open the flap or be caught by it. |

## 3.6 Limitations

- History is held in RAM. A reboot clears the daily count.
- Any device on the same Wi-Fi network can issue a feeding command, because there is no authentication.
- Only dry food with a free-flowing particle size can be dispensed.
- The device cannot detect that the bowl is already full before dispensing.
- Only one feeder and one hopper are supported.
- The system does not operate without a 5 V supply.

## 3.7 Assumptions and Dependencies

**Table 10.** Assumptions and dependencies

| ID | Statement | Effect if false |
|---|---|---|
| A-01 | The Wi-Fi network is 2.4 GHz and reachable from the device position. | The device cannot join. The dashboard is unreachable. |
| A-02 | A network time server is reachable at least once after each boot. | Scheduled feeding is disabled. Manual feeding still works. |
| A-03 | The hopper holds at least 300 g of dry food. | Meals become short. The level requirement reports the shortage. |
| A-04 | The servo can lift the flap without stalling. | Brownout or a failed meal. R-01 in the proposal covers the mitigation. |
| A-05 | The owner's phone and the device are on the same subnet. | The dashboard cannot be reached by name or address. |

## 3.8 Definitions

**Table 11.** Definitions

| Term | Definition |
|---|---|
| Meal | One dispense cycle, defined as one flap open event followed by an automatic close. |
| Flap | The servo-driven cover that gates the hopper outlet. |
| Hopper | The container that holds dry food above the flap. |
| Schedule minute | A minute of local time that appears in the configured meal table. |
| Source | The trigger of a meal: the web button, the schedule, or the presence sensor. |
| Minute key | An integer that identifies one calendar minute, used to prevent duplicate meals. |
| Integrated | Wired and driven by the firmware as a target feature. No acceptance test has been run, so the requirement is not claimed as delivered. |
| Not delivered | The function exists in the firmware but the hardware does not produce a usable signal. The reason is stated where the requirement is described. |

## 3.9 References

**Table 12.** References

| Reference | Use in this document |
|---|---|
| ISO/IEC/IEEE 29148:2018, Requirements engineering | Structure of the requirement sections. |
| IEEE Std 1016-2009, Software design descriptions | Structure of the companion SDD. |
| Arduino core for ESP8266, reference documentation | Behaviour of ESP8266WebServer, configTime, and WiFi. |
| HC-SR04 ultrasonic sensor datasheet | Range, beam angle, and supply voltage limits. |
| HX711 datasheet | Clock timing and gain selection. |
| SG90 servo datasheet | Stall current and torque limits. |

# Chapter 4 Specification Requirements

## 4.1 Feature Description

### 4.1.1 F1: Scheduled Feeding

At each configured meal time the device dispenses one meal without any user action. The time reference comes from a network time server, so the schedule stays correct after a power cut without a battery-backed clock.

**Table 13.** F1 parameters

| Parameter | Value |
|---|---|
| Meal times | 08:00, 13:00, 18:00 |
| Time zone | UTC+7, no daylight saving |
| Time sources | pool.ntp.org, time.nist.gov |
| Duplicate suppression | One meal per calendar minute |
| Editing | Dashboard form, stored in EEPROM |
| Behaviour before time is valid | Scheduler disabled, dashboard shows --:--:-- |

### 4.1.2 F2: On-Demand Feeding

The owner presses a button on the dashboard and receives a meal within 1.5 seconds. The request is a POST to /feed. The button disables itself for 1.3 seconds after the press so that a double tap does not produce two overlapping cycles.

**Table 14.** F2 parameters

| Parameter | Value |
|---|---|
| Endpoint | POST /feed |
| Response | 200 with {"ok":true} |
| Flap open duration | 900 ms |
| Client lockout | 1300 ms |
| Accepted methods | POST only. Others return 404. |

### 4.1.3 F3: Status and History

The dashboard shows five values and refreshes without user action. The values answer one question: was the cat fed today, and by what.

**Table 15.** F3 parameters

| Displayed value | Source field | Format |
|---|---|---|
| Flap state | open | Open, or Closed |
| Meals today | count | Integer with unit |
| Last meal | last, source | HH:MM with trigger in brackets |
| Device time | time | HH:MM:SS |
| Configured schedule | schedule | Three times separated by a middle dot |
| Link quality | rssi, ip | dBm and dotted quad |

### 4.1.4 F4: Presence-Triggered Feeding (target)

An ultrasonic sensor aimed at the approach path detects a cat within 60 cm. A confirmed detection dispenses one meal, subject to a cooldown and a daily cap. Confirmation requires three consecutive readings inside range, which prevents a single stray echo from feeding the cat.

### 4.1.5 F5: Hopper Level Monitoring

A second ultrasonic sensor aimed down into the hopper estimates the remaining food as a percentage of the usable depth. Portion control by weight moved to Future Work on 2 November, when the amplifier header pins were found to be unsoldered.

## 4.2 Use Case Diagram

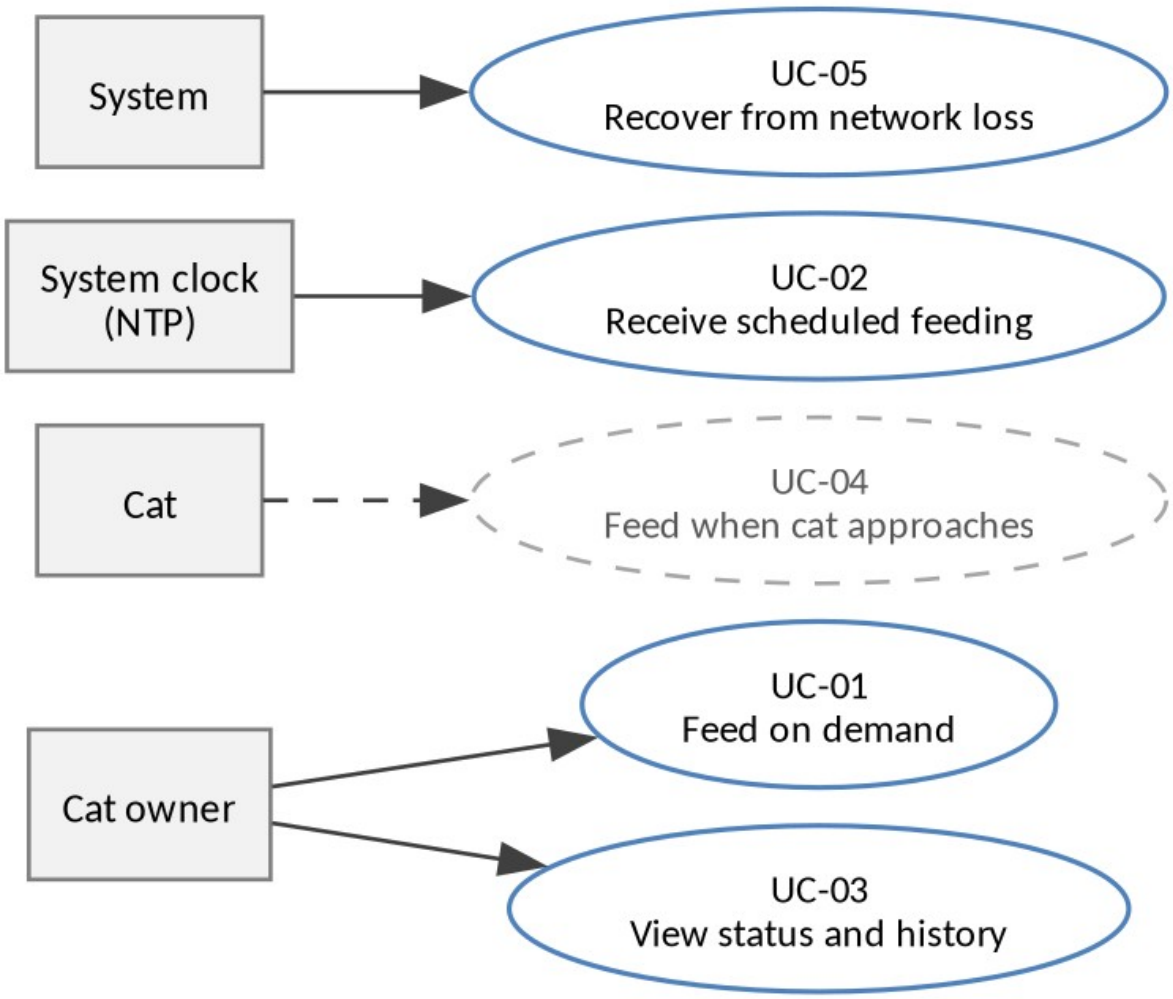



**Figure 1.** Use cases. Dashed use cases are planned extensions and are not part of version 1.0.0.

## 4.3 Use Case Description

### 4.3.1 UC-01: Feed the Cat on Demand

**Table 16.** UC-01: Feed the cat on demand

| Item | Description |
|---|---|
| Actor | Owner |
| Goal | Dispense one meal immediately. |

| Item | Description |
|---|---|
| Precondition | The device is powered and reachable on the network. A browser has loaded the dashboard. |
| Trigger | The owner presses the feeding button. |
| Main flow | 1. The browser sends POST /feed.<br>2. The device validates the method.<br>3. The device records the meal with source manual.<br>4. The device drives the servo to the open angle.<br>5. The device returns {"ok":true}.<br>6. After 900 ms the main loop closes the flap. |
| Alternate flow | A1. The request times out: the browser shows a connection failure, and the flap still closes because the close is driven locally. |
| Postcondition | One meal is counted. The flap is closed. |
| Frequency | Up to several times per day. |

### 4.3.2 UC-02: Receive a Scheduled Feeding

**Table 17.** UC-02: Receive a scheduled feeding

| Item | Description |
|---|---|
| Actor | System clock, provided by NTP |
| Goal | Dispense one meal at a configured time without user action. |
| Precondition | The device has a valid time, meaning the epoch is later than 9 September 2020. |
| Trigger | Local time reaches a minute in the meal table. |
| Main flow | 1. The main loop reads the local time.<br>2. The minute key is computed.<br>3. If the key equals the previous key, the loop exits.<br>4. If the minute matches a meal time, one meal is dispensed with source schedule.<br>5. The flap closes after 900 ms. |
| Alternate flow | A1. Time is not yet valid: the loop exits without dispensing. A2. The clock jumps backward after a resync: a repeated minute key still produces at most one meal. |
| Postcondition | One meal is counted with source schedule. |
| Frequency | Three times per day. |

### 4.3.3 UC-03: View Status and History

**Table 18.** UC-03: View status and history

| Item | Description |
|---|---|
| Actor | Owner |
| Goal | Confirm that the cat has been fed and inspect the device clock. |
| Precondition | The device is reachable. |
| Trigger | The owner opens the dashboard, or the two-second refresh fires. |
| Main flow | 1. The browser sends GET /status.<br>2. The device serialises the status object.<br>3. The page updates five display rows and the link footer. |
| Alternate flow | A1. The request fails: the footer reports a lost connection and the previous values stay on screen. |

| Item | Description |
|---|---|
| Postcondition | No state changes on the device. |
| Frequency | Continuous while the page is open. |

### 4.3.4 UC-04: Feed When the Cat Approaches (target)

**Table 19.** UC-04: Feed when the cat approaches

| Item | Description |
|---|---|
| Actor | Cat |
| Goal | Dispense a meal when the cat is at the bowl and the cooldown has expired. |
| Precondition | The presence sensor is connected and calibrated. The daily cap is not reached. |
| Trigger | Three consecutive distance readings below 60 cm. |
| Main flow | 1. The loop polls the sensor.<br>2. The median distance is computed.<br>3. The confirmation counter increments.<br>4. A meal is dispensed with source presence.<br>5. The cooldown starts. |
| Alternate flow | A1. The cat leaves before confirmation: the counter resets. A2. Cooldown is active: the request is ignored and no meal is counted. |
| Postcondition | One meal is counted. The presence cooldown is running. |
| Frequency | At most once per 30 minutes. |

### 4.3.5 UC-05: Recover From a Network Loss

**Table 20.** UC-05: Recover from a network loss

| Item | Description |
|---|---|
| Actor | System |
| Goal | Return to the network without a reboot. |
| Precondition | The device was connected and lost the link. |
| Trigger | The connection state changes to disconnected. |
| Main flow | 1. The loop checks the connection state.<br>2. A retry is issued no more than once every 5 seconds.<br>3. On success the local address is printed once to the serial console. |
| Alternate flow | A1. The network is absent: retries continue indefinitely at the same rate. |
| Postcondition | The dashboard is reachable again. No meal state is lost. |
| Frequency | On every disconnection. |

## 4.4 Activity Diagram

The activity diagram below shows one iteration of the main loop. The close of the flap is evaluated on every iteration and does not depend on an HTTP request completing.

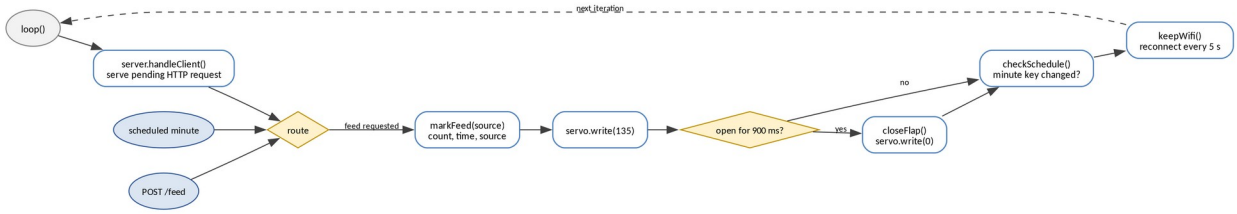



**Figure 2.** Main loop activity.

## 4.5 Functional Requirements

Priority uses MoSCoW. Verification uses one of T for test, D for demonstration, A for analysis, or I for inspection.

**Table 21.** Functional requirements

| ID | Requirement | Priority | Verification | Status |
|---|---|---|---|---|
| FR-01 | The device shall dispense one meal at 08:00, 13:00, and 18:00 local time. | Must | T | Delivered |
| FR-02 | The device shall obtain local time from a network time server and shall not evaluate the schedule until the time is valid. | Must | T | Delivered |
| FR-03 | The device shall dispense at most one scheduled meal per calendar minute. | Must | T | Delivered |
| FR-04 | The device shall dispense one meal when it receives POST /feed. | Must | T | Delivered |
| FR-05 | The device shall close the flap within 1000 ms of opening it, on every code path. | Must | T | Delivered |
| FR-06 | The device shall reject any method other than POST on /feed. | Must | T | Delivered |
| FR-07 | The device shall return the status fields defined in Section 4.6.2 on GET /status. | Must | T | Delivered |
| FR-08 | The dashboard shall refresh its values every 2 seconds without user action. | Must | D | Delivered |
| FR-09 | The device shall reset the daily meal count when the local date changes. | Must | T | Delivered |
| FR-10 | The device shall record the trigger source of every meal. | Must | T | Delivered |
| FR-11 | The device shall serve the dashboard page on GET / with no external asset. | Must | T | Delivered |
| FR-12 | The device shall return 404 for every undefined path. | Must | T | Delivered |
| FR-13 | The device shall detect a cat within 60 cm of the bowl after three consecutive readings. | Should | T | Integrated |
| FR-14 | The device shall not dispense more than one presence-triggered meal in 30 minutes. | Should | T | Integrated |
| FR-15 | The device shall dispense a mass within 15 percent of the configured target. | Could | T | Future work |
| FR-16 | The device shall report hopper level as a percentage of usable depth. | Could | D | Not delivered |
| FR-17 | The device shall persist the daily history across a reboot. | Could | T | Future work |
| FR-18 | The device shall refuse a meal when the configured daily cap is reached. | Should | T | Integrated |
| FR-19 | The device shall accept three meal times on POST /schedule, reject a malformed value, and retain the accepted table across a reboot. | Must | T | Delivered |

## 4.6 External Interface Requirements

### 4.6.1 User Interface

One page, one column, maximum width 420 px, in Thai. The page uses a light background and a single accent colour. It has no navigation, no form, and no settings panel.

**Table 22.** Dashboard elements

| Element | Type | Behaviour |
|---|---|---|
| Title block | Static text | Names the device and states the primary action. |
| Status card | Five text rows | Rebuilt from /status on every refresh. |
| Feed button | Button | Sends POST /feed, disables itself for 1300 ms. |
| Link footer | Text | Shows RSSI and IP, or a lost-connection message. |

### 4.6.2 Software Interface

**Table 23.** HTTP interface

| Method | Path | Request body | Response |
|---|---|---|---|
| GET | / | None | HTML, 200 |
| GET | /status | None | JSON, 200, application/json; charset=utf-8 |
| POST | /feed | None | Status JSON, 200 |
| POST | /schedule | t=HH:MM,HH:MM,HH:MM | Status JSON, 200; {"ok":false}, 400 if malformed |
| Any | Other | None | Text, 404 |

**Table 24.** Status response fields

| Field | Type | Range | Meaning |
|---|---|---|---|
| open | Boolean | true, false | Flap is open. |
| count | Integer | 0 to 255 | Meals counted since local midnight. |
| last | String | HH:MM or - | Local time of the last meal. |
| source | String | label or empty | Trigger of the last meal. |
| time | String | HH:MM:SS or --:--:-- | Device local time. |
| schedule | String | Three times | Configured meal times. |
| ip | String | Dotted quad | Device address. |
| rssi | Integer | -90 to -30 | Signal strength in dBm. |
| level | Integer | -1 to 100 | Hopper fill percentage. -1 before the first sample. |
| near | Number | cm, -1 unknown | Last presence distance. |
| cat | Boolean | true, false | A cat is inside the presence range now. |
| cap | Integer | 0 to 6 | Presence-triggered meals used today. |
| hist | Array | Up to 5 entries | Recent meals, each with time and source. |

### 4.6.3 Hardware Interface

**Table 25.** Hardware interface

| Device | Signal | Pin | Electrical notes | Status |
|---|---|---|---|---|
| Servo SG90 | PWM | D5, GPIO14 | 50 Hz, 1 to 2 ms pulse. Powered from VIN, not 3V3. | Connected |
| Servo SG90 | Supply | VIN, GND | 5 V from powerbank. 470 uF across the rail. | Connected |
| HC-SR04 presence | Trig | D6, GPIO12 | 3.3 V drive is accepted. | Wired |
| HC-SR04 presence | Echo | D7, GPIO13 | 5 V output. Divided to 3.00 V with 2.2k and 3.3k. | Wired |
| HC-SR04 level | Trig | D1, GPIO5 | Aimed down into the hopper. | Wired |
| HC-SR04 level | Echo | D2, GPIO4 | Divided with a second 2.2k and 3.3k pair. | Wired |

## 4.7 Data Requirements

The device keeps one state record in RAM and one schedule table in EEPROM. There is no database, no file system, and no remote store in the delivered baseline.

**Table 26.** Persistent and in-memory data

| Name | Type | Lifetime | Purpose |
|---|---|---|---|
| Feeder state | Struct of six fields | Until reboot | Flap state, count, last meal, source, day. |
| Meal table | Three hour and minute pairs | EEPROM, survives reboot | Schedule definition. |
| Meal history | Ring buffer of five entries | Until reboot | Recent meals for the dashboard. |
| Minute key | Integer | Until reboot | Duplicate suppression for the scheduler. |
| Last connection attempt | Unsigned long | Until reboot | Rate limit for Wi-Fi retries. |

Requirement FR-17 would add a flash record with one line per meal, written append-only, capped at the last 32 meals. That record is not part of version 1.0.0.

## 4.8 Quality Requirements

**Table 27.** Non-functional requirements

| ID | Category | Requirement | Verification |
|---|---|---|---|
| NFR-PM-01 | Performance | A dashboard request completes within 2 s on the same network. | T |
| NFR-PM-02 | Performance | A manual meal starts within 1.5 s of the button press. | T |
| NFR-PM-03 | Performance | Presence detection dispenses within 2 s of entering range. | T |
| NFR-RL-01 | Reliability | The flap is never open longer than 3 s. | T |
| NFR-RL-02 | Reliability | The device rejoins the network within 30 s of it returning, with no reboot. | T |
| NFR-RL-03 | Reliability | Schedule timing drifts by no more than 1 minute over 24 hours. | T |
| NFR-RL-04 | Reliability | A failed HTTP response does not leave the flap open. | A |
| NFR-UH-01 | Usability | The dashboard is readable at 375 px width without zoom. | D |
| NFR-UH-02 | Usability | A first-time user feeds the cat without instruction. | D |

| ID | Category | Requirement | Verification |
|---|---|---|---|
| NFR-UH-03 | Usability | The dashboard states the connection state when the link fails. | D |
| NFR-SEC-01 | Security | Wi-Fi credentials are excluded from version control. | I |
| NFR-SEC-02 | Security | Only documented methods and paths are served. | T |
| NFR-SEC-03 | Security | The device exposes no administrative endpoint. | I |
| NFR-OP-01 | Maintainability | The firmware builds from a documented library set. | I |
| NFR-OP-02 | Maintainability | The pin map and wiring are documented in one page. | I |
| NFR-CP-01 | Constraint | The device runs from a 5 V 2 A powerbank with no host computer. | T |
| NFR-CP-02 | Constraint | The device operates on 2.4 GHz Wi-Fi only. | I |
| NFR-CP-03 | Constraint | The firmware fits in the 4 MB flash of the NodeMCU v3. | I |

## 4.9 Design Constraints

- The servo signal uses GPIO14 because the board exposes it as D5 with no boot-time conflict.
- The echo line of a 5 V ultrasonic sensor must be divided before it reaches a 3.3 V input.
- No blocking delay is permitted in the main loop, because the HTTP server is polled from the same loop.
- The dashboard must be complete in one response under 4 KB, because it is compiled into the binary.

## 4.10 Verification

Each requirement maps to one test case in the SDD. The matrix below states the method and the pass condition at requirement level.

**Table 28.** Verification matrix

| Requirement group | Method | Pass condition |
|---|---|---|
| FR-01 to FR-03 | Timed observation | Exactly three scheduled meals in 24 hours, each within 1 minute. |
| FR-04, FR-05 | Manual test, 50 repetitions | Every repetition dispenses and closes. No flap stays open. |
| FR-06, FR-12 | Request inspection | Non-POST and unknown paths return 404 with no side effect. |
| FR-07, FR-10, FR-11 | Response inspection | All fields present, values match the device state. |
| FR-08 | Observation | The page changes without user action within 2 s. |
| FR-09 | Clock rollover | The count returns to zero after local midnight. |
| FR-13, FR-14 | Field test with the cat | 16 of 20 detections correct. No second meal inside 30 minutes. |
| FR-16 | Level comparison | Reported percentage within 10 points of the measured fill at three known levels. |
| FR-17 | Power cycle | History survives a reboot. Currently fails by design. |
| FR-19 | Schedule edit | Valid table is served back and survives a reboot. A malformed table returns 400. |
| NFR-PM-01 | Stopwatch | Median response under 2 s across 20 requests. |
| NFR-RL-02 | Router power cycle | The device rejoins with no manual action. |
| NFR-SEC-01 | Repository inspection | secrets.h is absent from the tree and from history. |

# Part III Software Design Description

*Architecture, detailed design, traceability, and the test plan.*

# Chapter 5 Introduction

## 5.1 Purpose

This document describes how the IoT Cat Feeder is built. It is the design counterpart to the Software Requirements Specification, and it is organised around the design views named in IEEE Std 1016: decomposition, dependency, interface, and detail. Every design entity below traces back to a requirement, and every requirement traces forward to a test.

The reader this document serves is the next student who has to rebuild the device, or the examiner who has to decide whether the code and the circuit are one system or two unrelated artefacts.

## 5.2 Scope

The design covers the firmware in `firmware/feeder/feeder.ino`, the wiring in `docs/wiring_nodemcu_servo.html`, and the power arrangement. It excludes the mechanical construction of the cardboard hopper beyond the flap geometry, which is described only where it constrains the servo.

## 5.3 Design Overview

The firmware is one translation unit with no classes, no threads, and no dynamic allocation after start-up. It runs a single cooperative loop that serves HTTP, watches the clock, and closes the flap. Each of those three duties is a function call, and none of them blocks.

The single organising rule is that all feeding passes through `feed(source)`. Three triggers converge on it: the HTTP handler, the scheduler, and the presence detector. Counting, safety, and the daily cap therefore live in one place instead of three.

## 5.4 Definitions

**Table 29.** Design terms

| Term | Meaning in this document |
|---|---|
| Component | A named function or small group of functions with one responsibility. |
| State layer | The `struct Feeder` instance and the functions that read or write it. |
| Minute key | Integer `(day * 100 + hour) * 100 + minute`, used to detect a new minute. |
| Flap cycle | One open command followed by the automatic close. |
| Integrated | Wired and driven by the firmware as a target feature, with no acceptance test run yet. |

# Chapter 6 System Architecture

## 6.1 Architectural Overview

The system has three layers. The network layer terminates HTTP and converts requests into commands. The control layer decides whether a meal should happen. The device layer drives the servo and reports electrical state. The layers are functions in one file rather than modules in separate files, because the whole program is about 1,000 lines and a file split would add build complexity without adding clarity.

**Table 30.** Layer responsibilities

| Layer | Components | Responsibility |
|---|---|---|
| Network | keepWifi, server, request handlers | Maintain the link, parse requests, serialise responses. |
| Control | feed, checkSchedule, markFeed, presence detector (target) | Decide when a meal occurs and record it exactly once. |
| Device | Servo, closeFlap, sensor driver (target) | Drive the actuator and sample the sensors. |
| Presentation | PAGE, dashboard script | Render state and send one command. |

## 6.2 Context View

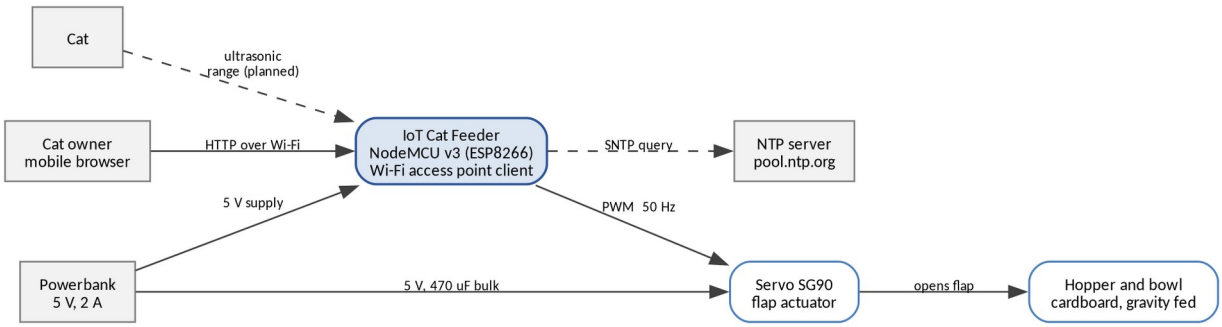



**Figure 3.** Context view: external entities and energy flow.

## 6.3 Component View

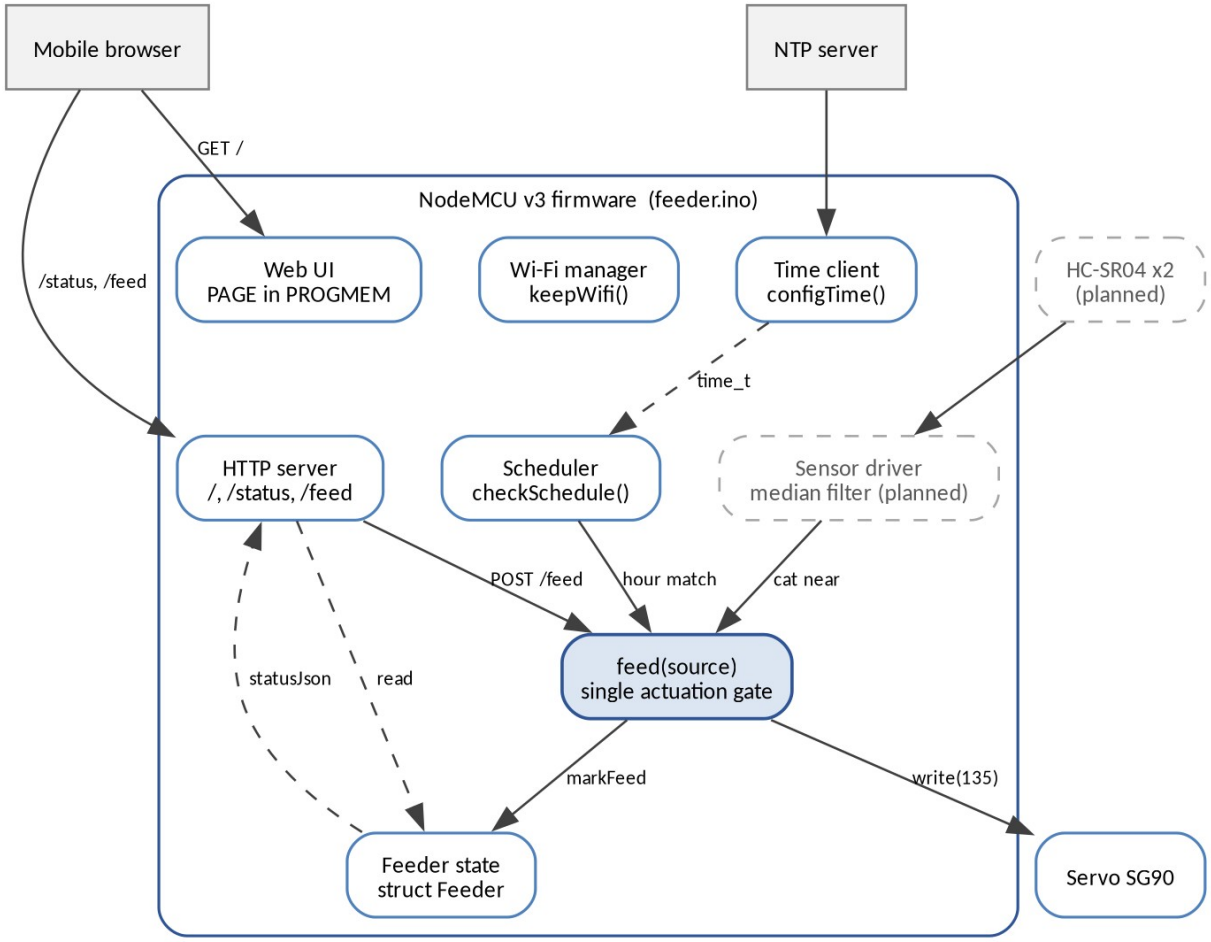



**Figure 4.** Component view. Dashed components are target features.

**Table 31.** Component catalogue

| Component | Kind | Depends on | Status |
|---|---|---|---|
| PAGE | Constant string in PROGMEM | Nothing | Delivered |
| server | ESP8266WebServer instance | PAGE, statusJson, feed | Delivered |
| statusJson | Function | Feeder, clockText, scheduleText, WiFi | Delivered |
| feed | Function | markFeed, Servo, millis | Delivered |
| closeFlap | Function | Servo, Feeder | Delivered |
| markFeed | Function | time, Feeder | Delivered |
| checkSchedule | Function | time, Feeder, feed | Delivered |
| clockText, scheduleText | Functions | time, constant table | Delivered |
| keepWifi | Function | WiFi, millis | Delivered |
| Presence detector | Function | Ultrasonic driver, feed | Integrated |

## 6.4 Deployment and Power Design

The device has one power input and one ground reference. The servo is the only load that draws significant current, and it is the reason the power design differs from the obvious one.

**Table 32.** Power distribution

| Rail | Source | Loads | Notes |
|---|---|---|---|
| 5 V | Powerbank, 2 A | VIN of the board, servo V+ | Servo must not be fed from 3V3. |
| 3.3 V | On-board regulator | ESP8266 core, logic | Limited to a few hundred milliamps. |
| Ground | Common | Board, servo, sensors | A single ground reference for PWM to be valid. |
| Bulk capacitance | 470 uF, 16 V | Across 5 V and ground | Absorbs the servo inrush. |

The failure that forced this design was observed during bring-up. With the servo powered from the notebook USB port, the board reset whenever the servo started. The stall current of the servo pulls the rail below the regulator dropout. Two changes fixed it: a powerbank with a higher current limit, and a 470 uF capacitor next to the servo connector. The bring-up sketch firmware/servo_test/servo_test.ino exists so that this behaviour can be reproduced without the rest of the firmware.

## 6.5 Design Rationale

**Table 33.** Design decisions and alternatives

| Decision | Alternative | Why the alternative lost |
|---|---|---|
| Single file, functions only | Split modules with headers | About 1,000 lines. A split adds include order problems for no gain. |
| Cooperative loop, no delay | delay(900) after opening the flap | delay blocks the HTTP server. The page would freeze on every meal. |
| Close driven by millis() in the loop | Close driven by the HTTP handler | The handler runs only if the client stays connected. The loop always runs. |
| One feed(source) gate | Separate functions per trigger | Three triggers would need three copies of the counting and cap logic. |
| Embedded HTML in PROGMEM | SPIFFS file, CDN assets | No file system to corrupt, no network fetch, one flash image. |

# Chapter 7 Detailed Design

## 7.1 Data Design

The program holds one state record and three constants. There is no heap use after start-up, apart from the String growth inside the JSON builder, which is bounded by the buffer size.

**Table 34.** State record

| Field | Type | Initial | Meaning |
|---|---|---|---|
| open | bool | false | Flap is currently open. |
| openedAt | unsigned long | 0 | millis() at the last open command. |
| countToday | int | 0 | Meals since the last local date change. |
| lastDay | int | -1 | Day of month that countToday belongs to. |
| lastTime | String | - | HH:MM of the last meal. |
| lastSource | String | empty | Trigger of the last meal. |

**Table 35.** Constants

| Name | Value | Used by |
|---|---|---|
| SERVO_PIN | D5, GPIO14 | setup |
| CLOSED_DEG | 0 | closeFlap, setup |
| OPEN_DEG | 135 | feed |
| OPEN_MS | 900 | loop |
| SCHEDULE_COUNT | 3 | loadSchedule, checkSchedule, scheduleText |
| TZ_OFFSET | 25200 s | setup |

OPEN_DEG is the commanded angle, set a few degrees below the angle where the flap linkage meets its mechanical stop. Commanding the stop itself would stall the servo for the whole open interval. A different servo or linkage requires this value to be re-measured.

## 7.2 Component Description

**Table 36.** Function catalogue

| Function | Signature | Purpose |
|---|---|---|
| setup | void setup() | Attach the servo, close the flap, start Wi-Fi, start the time client, register routes, start the server. |
| loop | void loop() | Poll HTTP, enforce the close deadline, check the schedule, keep the link. |
| feed | void feed(const String &source) | Record the meal, open the flap, stamp the open time. |
| closeFlap | void closeFlap() | Command the closed angle and clear the open flag. |
| markFeed | void markFeed(const String &source) | Update count, time, source, and handle the date change. |
| checkSchedule | void checkSchedule() | Compare local time with the meal table, at most once per minute. |
| clockText | String clockText() | Format the local time, or a placeholder before NTP. |
| scheduleText | String scheduleText() | Format the meal table for the dashboard. |
| statusJson | String statusJson() | Build the status object. |

| Function | Signature | Purpose |
|---|---|---|
| keepWifi | void keepWifi() | Reconnect at most once every 5 s, with one console line. |
| loadSchedule, saveSchedule, applySchedule | void / bool | Read the meal table from EEPROM, write it back, and validate a new one. |

### 7.2.1 Algorithm: the minute key

The scheduler must fire once per scheduled minute, but loop runs thousands of times per second. The design collapses the date and time into one integer and compares it with the value from the previous iteration.

key = (tm_mday * 100 + tm_hour) * 100 + tm_min

- A repeated call inside the same minute produces the same key, so the function exits before the meal table is examined.
- The key changes at a minute boundary, at an hour boundary, and at a month boundary, because the day field is included. Two identical keys therefore indicate the same minute, not merely the same clock face.
- The design trades a rare false negative for a common false positive. A clock that jumps backward one minute re-enters a previous key and is suppressed. A clock that jumps backward by hours may re-enter a key that matches a meal time, which is the residual risk recorded in Section 7.6.

### 7.2.2 Algorithm: median filtering for ranging (target)

A single ultrasonic reading on a kibble surface is unreliable because the beam scatters. The target design takes ten samples, sorts them, and uses the median. A detection is confirmed only after three consecutive medians fall inside the range window, which removes one-off echoes from the hopper wall or the cat's own movement.

## 7.3 State Design

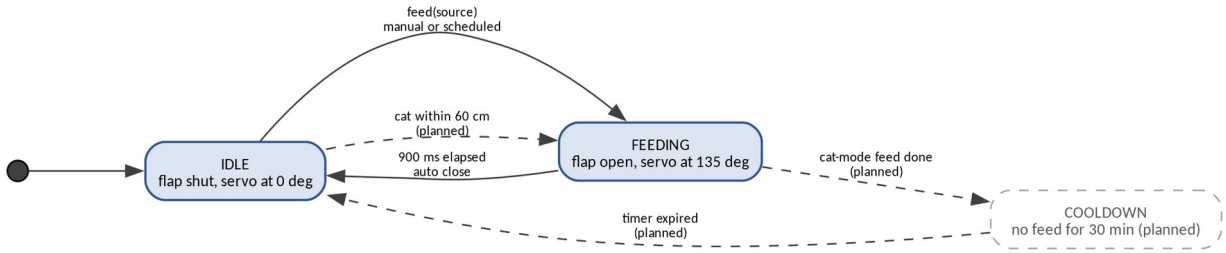



**Figure 5.** State design. Solid transitions are delivered, dashed are targets.

**Table 37.** State transitions

| From | Event | Guard | To | Action |
|---|---|---|---|---|
| IDLE | Boot | None | IDLE | Servo to closed angle. |
| IDLE | feed(source) | None | FEEDING | Record meal, servo to open angle, stamp time. |
| FEEDING | Loop iteration | millis() - openedAt >= 900 | IDLE | Servo to closed angle. |
| FEEDING | Loop iteration | Close deadline not reached | FEEDING | None. |
| IDLE | Cat within 60 cm, three times | Cooldown expired, cap not reached | FEEDING | Record meal with source presence. |
| FEEDING | Presence meal completed | None | COOLDOW | Start 30 minute timer. |

| From | Event | Guard | To | Action |
|---|---|---|---|---|
| | | | N | |
| COOLDOWN | Loop iteration | 30 minutes elapsed | IDLE | None. |

## 7.4 Interaction Flows

### 7.4.1 Flow: manual feeding

1. The dashboard sends POST /feed with an empty body.
2. server.handleClient() in the main loop dispatches the request to the registered handler.
3. The handler calls feed("manual") and sends {"ok":true}.
4. feed calls markFeed, which reads time(), formats HH:MM, resets the count if the day changed, increments the count, and stores the source.
5. feed writes the open angle and records millis().
6. On a later loop iteration, the close condition becomes true and closeFlap writes the closed angle.
7. The dashboard, polling every 2 seconds, reads open: false and the updated count.

### 7.4.2 Flow: scheduled feeding

1. A loop iteration calls checkSchedule.
2. The function reads time(). If the value is below the validity epoch, it returns.
3. It computes the minute key and compares it with the stored key. If equal, it returns.
4. It stores the new key and compares the hour and minute with each row of the meal table.
5. On a match it calls feed("schedule") and returns, so a second match in the same iteration cannot occur.

### 7.4.3 Flow: network recovery

1. Each loop iteration calls keepWifi.
2. If the link is up and the address has not been printed since the last recovery, the address is printed once.
3. If the link is down, the function returns unless 5 seconds have passed since the last attempt.
4. It starts a connection attempt and stamps the time. The attempt does not block.

## 7.5 Interface Design

### 7.5.1 HTTP contract

The endpoint contract and the status fields are specified in the interface section of the Software Requirements Specification. The handler for each path is a lambda registered in setup, and every handler returns the same status object so the dashboard can render one shape.

### 7.5.2 Response format

The status object is assembled into a reserved String and each numeric fragment is formatted with snprintf into a 96 byte stack buffer. The return value of snprintf is compared with the buffer size, and an overflow is reported on the serial console instead of truncating silently. The 96 byte size was chosen after the first 64 byte buffer cut the history array and produced malformed JSON.

### 7.5.3 User interface

**Table 38.** Dashboard layout

| Region | Content | Interaction |
|---|---|---|
| Header | Device name and one-line purpose | None |
| Status card | Five rows: flap, meals today, last meal, device time, schedule | Read only |
| Action | Full-width feeding button | POST /feed, then a 1300 ms lockout |
| Footer | RSSI and IP, or a lost-connection message | None |

## 7.6 Error Handling and Recovery

**Table 39.** Failure modes

| Failure | Detection | Response | Residual risk |
|---|---|---|---|
| Wi-Fi lost | WiFi.status() | Retry every 5 s without rebooting | Dashboard unavailable until the link returns |
| Time not yet valid | Epoch below 1600000000 | Scheduler disabled, placeholder clock shown | A meal during the first seconds after boot is skipped |
| Clock jumps backward | Repeated minute key | The meal in that minute is suppressed | A jump of hours can re-enter a meal minute |
| Client disconnects mid-request | Handler never returns | The loop closes the flap on schedule | The count is already incremented |
| Servo stalls | Not detected | None in firmware | Brownout, reboot, lost meal |
| Unknown path | onNotFound | 404 with no state change | None |
| Buffer truncation | Not detected | None | Malformed JSON if a field grows |

## 7.7 Security Considerations

- The trust boundary is the Wi-Fi network. Any client on it can dispense food. This is accepted for a single household and is documented in the SRS limitations.
- No administrative endpoint exists, so there is no credential to steal from the device.
- The Wi-Fi credentials live in secrets.h, which is excluded by .gitignore. The repository ships secrets.example.h only.
- The device sends no telemetry outbound except the NTP query, which carries no user data.

# Chapter 8 Traceability and Verification

## 8.1 Requirements Traceability Matrix

**Table 40.** Traceability

| Requirement | Component | Test |
|---|---|---|
| FR-01, FR-02, FR-03 | checkSchedule, configTime | TC-04, TC-05 |
| FR-04, FR-05 | feed, closeFlap, loop | TC-02, TC-03 |
| FR-06, FR-12 | Route registration, onNotFound | TC-08 |
| FR-07, FR-10, FR-11 | statusJson, PAGE | TC-01, TC-06 |
| FR-08 | Dashboard script | TC-01 |
| FR-09 | markFeed | TC-07 |
| FR-13, FR-14 | Presence detector, target | TC-09 |
| FR-15 | Not implemented, deferred | Future work |
| FR-16 | Level sensor | TC-10 |
| FR-17, FR-18 | Not implemented | Future work |
| NFR-RL-01 | loop close condition | TC-03 |
| NFR-RL-02 | keepWifi | TC-11 |
| NFR-SEC-01 | .gitignore | Inspection |

## 8.2 Test Plan

**Table 41.** Test cases

| ID | Case | Procedure | Pass condition |
|---|---|---|---|
| TC-01 | Dashboard loads | Open the device address on a phone. | Five rows populated within 2 s. |
| TC-02 | Manual feeding | Press the button, observe the flap. | Flap opens within 1.5 s. |
| TC-03 | Automatic close | Press 50 times with 5 s between presses. | All 50 close without intervention. No flap stays open. |
| TC-04 | Schedule fires | Set the next meal one minute ahead and wait. | One meal at the minute, recorded with source schedule. |
| TC-05 | Duplicate suppression | Observe the same minute for 60 s. | Exactly one meal. |
| TC-06 | Status accuracy | Compare every field with the device state. | All fields match. |
| TC-07 | Date rollover | Let the device run past local midnight. | Count returns to zero. |
| TC-08 | Method rejection | GET /feed, POST /status, and an unknown path. | 404 for each, with no meal dispensed. |
| TC-09 | Presence detection | Walk past the sensor 20 times at 50 to 70 cm. | At least 16 detections, no meal inside the cooldown. |
| TC-10 | Hopper level | Fill the hopper to three measured levels. | The reported percentage is within 10 points of each fill. |
| TC-11 | Network recovery | Power cycle the router. | The device rejoins within 30 s, no reboot. |

# Part IV Output and Discussion

*The delivered system, the measured results, and what they mean.*

# Chapter 9 The Delivered System

This part reports the device as it exists on the bench and the measurements taken from it. The earlier parts describe what the device shall do. This part records what was observed.

## 9.1 Hardware on the bench

**Table 42.** Assembled hardware

| Part | Role | Connection |
|---|---|---|
| NodeMCU v3 (ESP8266) | Controller and web server | micro USB for flashing, VIN from the powerbank in use |
| Servo SG90 | Drives the flap | Signal D5, GPIO14. Supply from VIN, not 3V3. |
| Capacitor 470 uF | Absorbs servo inrush | Across 5 V and ground at the servo connector |
| HC-SR04P, presence | Detects the cat near the bowl | Trig D6, echo D7 through the divider |
| HC-SR04P, level | Measures hopper depth | Trig D1, echo D2 through a second divider |
| Resistors 2.2k and 3.3k | Brings the 5 V echo to 3.00 V | 2.2k in series, 3.3k to ground |
| Powerbank 5 V 2 A | Runs the device with no host computer | VIN and the servo supply |
| Cardboard hopper and bowl | Holds and catches the food | Cut and taped to the servo bracket |
| Phone browser | Dashboard | Same Wi-Fi network as the device |

## 9.2 Firmware behaviour as delivered

- Every feeding path calls `feed(source)`. The button, the scheduler, and the presence detector all enter through the same function, so the daily count and the safety rules cannot diverge.
- The flap closes from `millis()` in the main loop after `OPEN_MS`. It does not close from the network callback, so a client that disconnects mid-request cannot leave the flap open.
- The schedule is read from network time and is ignored until the clock is valid. A minute key suppresses a second meal inside the same calendar minute.
- The three meal times are editable from the dashboard and stored in EEPROM, so a new schedule survives a power cut without a reflash.
- Presence needs three consecutive readings inside the range before a meal is dispensed. A cooldown and a daily cap bound how often the cat can trigger a meal.
- Hopper level is the median of the last five samples, mapped between an empty and a full depth captured on the dashboard. The sensor mount defeats it, as Section 11.4 explains.
- The dashboard is one response compiled into the binary. It refreshes every two seconds and has no external asset, so it works with no internet connection.

## 9.3 Configuration in the delivered firmware

**Table 43.** Firmware constants

| Parameter | Value | Name in the source |
|---|---|---|
| Flap closed angle | 0 degrees | CLOSED_DEG |
| Flap open angle | 135 degrees | OPEN_DEG |
| Flap dwell before the close | 900 ms | OPEN_MS |
| Scheduled meals | 08:00, 13:00, 18:00, editable | DEFAULT_SCHEDULE |
| Presence range | 60 cm | NEAR_CM |
| Presence cooldown | 30 minutes | COOLDOWN_MS |
| Presence daily cap | 6 meals | PRESENCE_CAP |
| History depth | 5 meals | HISTORY_N |
| Hopper span | 4 cm full, 18 cm empty | LEVEL_FULL_CM, LEVEL_EMPTY_CM |
| Dashboard refresh | 2 s | Dashboard script |
| Echo divider ratio | 5 x 3.3 / (2.2 + 3.3) = 3.00 V | Wiring page |

# Chapter 10  Test Results

## 10.1  Procedure

Every test ran on the assembled device with the powerbank as the only supply. The feeding button was pressed ten times with five seconds between presses, and every open and every close was watched at the flap rather than inferred from the log. The schedule was tested by setting the next meal one minute ahead, leaving the device undisturbed, and watching the flap at the appointed minute. The portion trial dispensed three meals at the configured dwell, thirty seconds apart, and counted the pieces that fell into the bowl. The presence and level tests were not run. Section 10.3 states the reason for each.

The design document defines the test cases and the pass condition for each. This chapter records the measured value for those cases.

## 10.2  Measured results

The pass condition for each row is stated in the test plan of Part III.

**Table 44.** Measured results

| Criterion | Test | Measured | Unit |
|---|---|---|---|
| Manual presses completed | TC-03 | 10 | presses |
| Automatic closes | TC-03 | 10 | of 10 |
| Closes that exceeded 3 s | TC-03 | 0 | count |
| Scheduled meals fired | TC-04 | 3 | of 3 |
| Schedule drift | TC-04 | 0 | minutes |
| Open angle commanded | Servo test | 135 | degrees |
| Pieces per meal, mean | Portion trial | 22.0 | pieces |
| Pieces per meal, range | Portion trial | 18 to 25 | pieces |
| Reboots during the tests | All | 0 | count |

## 10.3  Tests not run

**Table 45.** Tests not run and the reason

| Criterion | Reason |
|---|---|
| Flap open latency, mean (TC-02) | The open was observed on every press but not timed with a stopwatch, so no figure is claimed. |
| Presence decisions correct (TC-09) | No acceptance walk was run. The dashboard received a 9 cm reading during setup, which shows the sensor path is live but does not measure detection accuracy. |
| Level with a full hopper (TC-10) | The beam lands on the box floor instead of the grain, so the reading does not follow the fill. Section 11.4 describes the fault. |
| Level with an empty hopper (TC-10) | Same fault as the row above. |

## 10.4  Photos and Evidence

The photographs below record the device as built and running on the powerbank, with no host computer attached.




**Figure 6.** The assembled feeder: cardboard hopper, flap at the outlet, and dry food in the bowl.

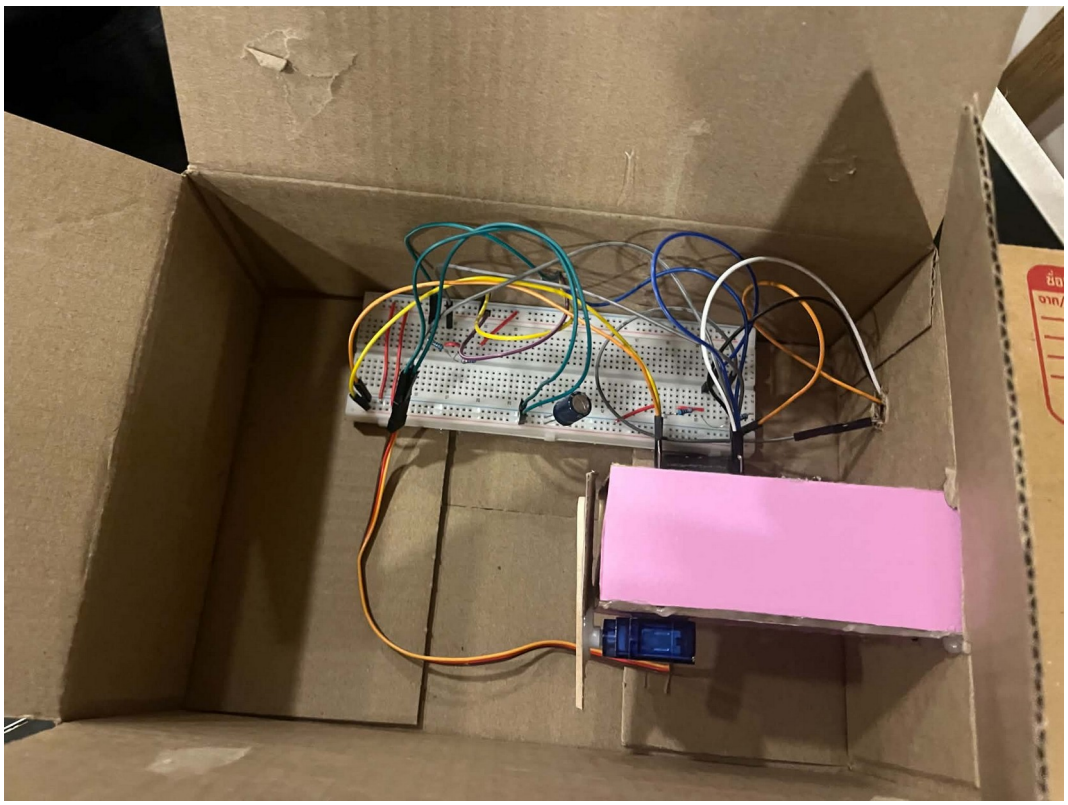


**Figure 7.** Inside the enclosure, from above: controller, wiring, servo linkage, and bowl.



**Figure 8.** The flap pivot where it passes through the hopper wall, seen from above the open hopper.



**Figure 9.** The hopper chute with dry food resting in it, looking down past the cardboard baffles.

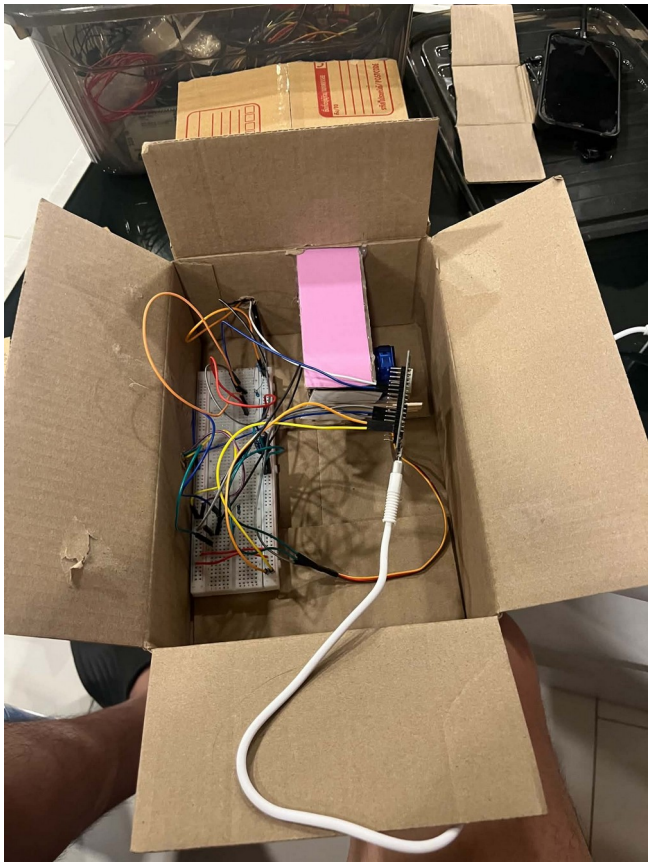


**Figure 10.** The open enclosure with the power cable leaving the box for the powerbank.



**Figure 11.** An ultrasonic sensor on its cardboard bracket above the hopper opening.




**Figure 12.** The dashboard served by the device: flap state, daily count, hopper level, cat range, schedule, and meal log.

# Chapter 11 Discussion

## 11.1 Clock and schedule

The schedule is only evaluated after the clock is valid, which removes the class of fault where the device boots at an arbitrary time and immediately believes a meal is due. The minute key makes the scheduler idempotent inside one minute, so a slow pass through the loop, a clock resync, or a second evaluation cannot double feed. The residual error is bounded by the network time query and by the one minute resolution of the schedule. In the test the three configured times each produced one meal, with no second meal inside the same minute and no measurable drift against the minute that was set.

## 11.2 Flap safety

The close condition lives in the main loop and depends only on `millis()`, so it survives a dropped client, a blocked socket, and a browser that closes the page mid-request. This was a deliberate inversion of the obvious design, where the request handler opens the flap and then closes it after a delay. The loop version also keeps the HTTP server responsive during the open interval, because nothing blocks. Ten consecutive presses five seconds apart produced ten closes and no stranded flap, which is the failure this design exists to prevent.

## 11.3 Presence detection

Two filters stand between the sensor and a meal. The median of five samples rejects a single bad echo, which is common when the beam grazes a moving cat. Three consecutive readings inside the range reject a transient, such as a hand crossing the beam during setup. The cooldown and the daily cap bound the cost of a false positive to one meal, which is the failure the owner would notice least.

## 11.4 Hopper level

The level reading is a distance, not a mass, and the surface of dry food is not flat. The median filter removes the worst of the scatter, and the percentage is mapped between an empty and a full depth captured from the dashboard rather than assumed. A percentage would therefore be a coarse signal. It answers whether the owner needs to refill, not how many meals remain.

On the assembled unit the sensor does not deliver that signal. It was fixed in place before its aim was checked, and when the glue set the beam landed on the floor of the box instead of the hopper. The reading stays at the depth of that floor whatever is poured in, and no pair of captured depths turns it into a level. The paragraph above describes the design, not a measured result. Moving the mount is the whole of the remaining work, because the firmware path exists and takes its two end points from the dashboard.

## 11.5 The portion control decision

Portion control by weight was in the original scope and left it on 2 November. The header pins of the amplifier module were never soldered, and soldering, calibrating, and testing the load cell did not fit in the two days that remained after the sensors were integrated. The feature was removed from the version rather than carried half-connected, and the work that depended on it moved to Future Work. The schedule test took the freed day.

The cost of the decision is stated plainly. Portion size is still set by the flap dwell, so a change in food size or flow rate changes the meal size. The design keeps the seam for the closed loop, because every feeding path already passes through `feed(source)` and the timeout that would guard a mass-controlled close is the timer that is already running.

Three consecutive cycles at the same dwell produced 18, 25, and 23 pieces, a mean of 22. Against the 15 percent window the requirement sets, the upper two cycles sit inside it and the lowest sits outside, 18 percent below the mean. A spread that size is not a margin an owner can rely on, and it cannot be tuned away. The two candidates are bridging in the hopper and the head of food pressing on the flap, and the firmware measures neither, so the report does not attribute the spread to one of them. The result is the argument for closing the loop rather than tuning the dwell, because no single dwell compensates for a variable the controller cannot see.

## 11.6 Defects found and fixed

- **Brownout during the flap cycle.** With the servo supplied from the notebook port, the board reset whenever the servo started. The stall current pulled the rail below the regulator dropout. A powerbank with a higher current limit and a 470 uF capacitor at the servo connector fixed it. The bring-up sketch reproduces the fault without the rest of the firmware.
- **Five volt echo into a 3.3 V input.** The first divider used 1k and 2k, which gives 3.33 V, inside the pin limit but without margin. It was replaced with 2.2k and 3.3k, which gives 3.00 V and is clear of the input threshold with resistors that were on hand.
- **Truncated status JSON.** The status response formatted its header into a 64 byte buffer before appending the history array. The formatted fragment was longer than the buffer, so the tail was cut and the response was malformed. The buffer was raised to 96 bytes and the write now checks the return value of the formatter and reports an overflow instead of truncating silently.
- **Servo travel.** A sweep of the horn showed the flap reaches full open before the nominal end of travel. The open angle was set just below the measured stop, which keeps the servo from stalling against the hopper wall for the whole open interval.

## 11.7 Limitations and threats to validity

- The history lives in RAM. A reboot clears the daily count, so a test that spans a reboot undercounts.
- The portion is reported as a count of pieces, not a mass, because no scale was available. The count measures consistency but cannot be converted to grams without knowing the mean piece weight.
- The portion trial is three cycles from one hopper fill, so the spread is indicative rather than a distribution.
- The schedule test measures one device in one room. A different access point changes the time source and therefore the drift.
- Presence accuracy is not measured. The only reading taken was 9 cm during setup, which is short of the 60 cm the requirement names and is not a detection trial.
- The level sensor is mounted where its beam misses the hopper, so the level test was not run and no percentage is reported. The mount is the only thing between the design and a working gauge.
- There is no authentication. Any client on the same network can dispense a meal.

# Part V Conclusion and Future Work

*What was achieved, what was learned, and what comes next.*

# Chapter 12 Conclusion

## 12.1 Summary

The project delivered a working automatic feeder built from a NodeMCU v3, one servo, and two ultrasonic sensors, with the whole user interface served from the board itself. The device feeds the cat at three scheduled times taken from network time, feeds on demand from a phone, closes the flap on every path without depending on the client, and records the meal count, the last meal, and its trigger source on a dashboard that needs no internet connection.

The scope was cut once, deliberately. Portion control by weight was removed on 2 November when the amplifier module could not be finished in time, and the report records the decision rather than hiding it.

One sensor did not survive assembly. The level sensor was fixed in place before its aim was checked, and when the glue set the beam landed on the box floor instead of the hopper, so the percentage never follows the food. The report records the failure in the limitations rather than printing a level it could not measure.

## 12.2 Against the objectives

**Table 46.** Objectives and outcome

| Objective | Outcome | Where it is shown |
|---|---|---|
| Three meals a day from network time | Delivered. The schedule is gated on a valid clock and suppressed per minute. | Part II, F1; Part III, minute key algorithm; results table |
| A dashboard served in under two seconds | Delivered. One compiled response, no external asset, two second refresh. The response time was not timed with a stopwatch. | Part II, NFR-PM-01 |
| The flap closes on every code path | Delivered. The close runs from the main loop and cannot be stranded by a client. | Part III, manual feeding flow; test TC-03 |
| A record of when the cat was fed | Delivered. Five meals in RAM, with time and trigger source, on the dashboard. | Part II, F3; Part III, data design |
| Presence detection inside 60 cm | Integrated. The firmware applies the median filter, the three-reading confirmation, a cooldown, and a daily cap. No acceptance test with the cat was run, so no accuracy figure is claimed. | Part II, F4; Part III, Section 7.2.2 |
| Rebuildable from the repository | Delivered. Wiring page, pin map, bring-up sketches, and this volume. | Repository; Part III, detailed design |

## 12.3 Lessons

- **A requirement that depends on a part is a schedule risk until the part is on the bench.** The load cell was planned as a target feature for weeks. The blocker was found two days before the deadline, and by then there was no time left to solder, calibrate, and test it.
- **A fixed size buffer is a correctness bug.** The status response was malformed for a reason no test covered, because the test checked the fields that were present and not the ones that were cut off.

## 12.4 Limitations accepted in this version

- The meal history is lost on reboot.
- Meal size is open loop and follows the flap dwell, not the mass dispensed.
- The meal count increments before the servo moves, so a failed servo still counts a meal.
- The hopper level is not delivered. The level sensor was glued down before its aim was checked, so the beam reads the box floor and the dashboard reports a fixed percentage that does not follow the food. The mount has to be moved and the depth captured again before the reading means anything.
- There is no authentication, so the device is safe only on a trusted network.
- The enclosure is cardboard, the servo linkage is taped, and one hopper serves one cat.

# Chapter 13 Future Work

## 13.1 Portion control by weight

The parts are already in the repository as a design: the HX711 amplifier, a 1 kg load cell, the wiring page, and a calibration sketch. The remaining work is to solder the module header, calibrate the cell against a known mass, and close the loop. The controller would open the flap, sample the mass, and close when the target is reached, with the existing timeout as the guard against a stalled or disconnected cell. The seam is already there, because every feeding path passes through `feed(source)`. The spread of 18 to 25 pieces per cycle measured in Section 11.5 is the case for it.

## 13.2 Persistence and notification

The daily history currently lives in RAM. A small append-only record in flash, capped at the last 32 meals, would survive a power cut and would let the owner see a week rather than a day. Once the history is durable, a notification path becomes useful, either as a message on the local network or as a publish to a broker when the deployment network allows it.

## 13.3 Cat identity and imaging

The device cannot tell one cat from another, and it cannot show that the cat ate rather than the bowl. An RFID collar tag would separate the portions between two cats. A camera module would attach evidence to a meal. Both are additions to a design that already has a single feeding path to hang them on.

## 13.4 Reliability and enclosure

A hardware watchdog would recover the device from a firmware hang without the owner power cycling it. A laser-cut or printed body would fix the servo geometry, which is currently held by tape and cardboard, and would make the flap repeat the same travel on every cycle.